

FinTrack — Angular Frontend Implementation Guide

Purpose: Step-by-step guide to create the FinTrack Angular frontend against the existing FinTrackCore API.

Backend repo: `FinTrackCore.slnx` (ASP.NET Core, Clean Architecture, PostgreSQL)

API base (dev): `http://localhost:5027` or `https://localhost:7015`

1. Backend analysis (first → last)

1.1 What FinTrack is

FinTrack is a **multi-user personal finance app** with **double-entry accounting** on the backend. The UI stays simple (Income / Expense / Transfer); the API creates balanced vouchers internally (not built yet).

Each user owns all financial data. Data is isolated by `userInfoId` from JWT.

1.2 Backend stack (done)

Layer	Technology
API	ASP.NET Core 10
Architecture	Clean Architecture (Domain → Application → Infrastructure → Api)
Database	PostgreSQL + EF Core Code-First
Auth	JWT access (15 min) + refresh token (7 days, rotation, DB)
Response	Unified <code>ApiResponse<T></code> envelope
Messages	<code>MessageSettings</code> class (not appsettings)

1.3 Backend features — completed

#	Feature	Status	Notes
1	User register (manual)	Done	<code>POST /api/UserInfos</code> — seeds default COA
2	Login (password)	Done	<code>POST /api/Auths/login</code>
3	Google signup/login	Done	<code>POST /api/Auths/google</code> — needs <code>Google:ClientId</code>
4	Refresh token	Done	<code>POST /api/Auths/refresh</code> — rotation
5	Logout	Done	<code>POST /api/Auths/logout</code> — revokes refresh
6	Authorize + user isolation	Done	JWT Bearer; user can only access own data
7	AccountType (lookup)	Done	Seeded: ASSET, LIABILITY, EQUITY, INCOME, EXPENSE
8	COA (Chart of Accounts)	Done	Per-user tree; 18 default heads on register
9	Profile get/update	Done	<code>GET/PUT /api/UserInfos/{id}</code>

1.4 Backend features — not started (Angular will stub UI)

#	Feature	Needed for dashboard
1	FinancialYear	Posting period
2	TransactionType	INCOME, EXPENSE, TRANSFER, etc.
3	Transaction + Voucher	Real money movements
4	Reports (Dapper)	Monthly cash flow, balances
5	LoginLog	Optional security history

Dashboard v1 can ship with **profile + COA + auth**; transaction widgets use placeholders until backend Phase 2.

1.5 API response envelope (all endpoints)

Every API returns:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Login successful.",
  "data": { },
  "meta": null
}
```

Paged lists (future) will use:

```
"meta": { "totalData": 100, "totalPage": 10 }
```

Angular must always read **data** from the envelope, not the raw body.

1.6 Auth flow (frontend must implement)

Register (manual OR Google)

- store accessToken + refreshToken + user
- attach Bearer on every API call

Access token expires (401)

- POST /api/Auths/refresh with refreshToken
- retry original request with new accessToken
- if refresh fails → clear storage → redirect /login

Logout

- POST /api/Auths/logout with refreshToken
- clear local storage → redirect /login

Do **not** poll refresh every N minutes. Refresh only on **401** or shortly before expiry.

1.7 Current API map

Method	Route	Auth	Body / Response data
--------	-------	------	----------------------

POST	/api/UserInfos	Anonymous	Register → { id }
GET	/api/UserInfos/{id}	Bearer	UserInfoResponse
PUT	/api/UserInfos/{id}	Bearer (own id)	{ id }
POST	/api/Auths/login	Anonymous	{ accessToken, refreshToken, expiresIn, user }
POST	/api/Auths/google	Anonymous	Same as login
POST	/api/Auths/refresh	Anonymous	Same as login
POST	/api/Auths/logout	Anonymous	message only
GET	/api/AccountTypes	Bearer	AccountType[]
GET	/api/Coas	Bearer	Coa[]
GET	/api/Coas/{id}	Bearer	Coa
POST	/api/Coas	Bearer	{ id }
PUT	/api/Coas/{id}	Bearer	{ id }
DELETE	/api/Coas/{id}	Bearer	{ id }
GET	/api/Healths	Anonymous	health info

1.8 Default COA (auto-created on register)

After register/login, user already has 18 system accounts, e.g.:

- **Assets:** Cash (10100), Bank (10200), Mobile Wallet (10300)
- **Liabilities:** Credit Card (20100)
- **Income:** Salary (40100), Freelance (40200)
- **Expenses:** Food, Transport, Rent, Utilities, Shopping, Entertainment

Angular COA screen shows tree built from `parentId` .

2. Angular project — create & configure

2.1 Prerequisites

- Node.js 20 LTS+
- Angular CLI 19+ (`npm i -g @angular/cli`)
- Backend running on `http://localhost:5027`

2.2 Create project

From repo root (`D:\Hridoy\FinTrack`):

```
ng new fintrack-web --routing --style=scss --ssr=false --standalone
cd fintrack-web
```

Recommended additions:

```
ng add @angular/material
npm install @angular/google-signin # or use GIS script for Google button
```

2.3 Suggested folder structure (mirror backend features)

```
src/app/
├── core/                                # Singleton services, guards, interceptors
│   ├── auth/
│   │   ├── auth.service.ts
│   │   ├── auth.guard.ts
│   │   ├── guest.guard.ts
│   │   └── token-storage.service.ts
│   ├── http/
│   │   ├── api.service.ts             # unwrap ApiResponse<T>
│   │   ├── auth.interceptor.ts        # Bearer + refresh on 401
│   │   └── error.interceptor.ts
│   ├── models/
│   │   ├── api-response.model.ts
│   │   └── auth.model.ts
│   └── constants/
│       └── api-endpoints.ts
├── shared/                              # Reusable UI (buttons, tables, loaders)
│   ├── components/
│   └── pipes/
│       └── currency.pipe.ts           # uses user.currencyCode
├── features/
│   ├── auth/
│   │   ├── login/
│   │   ├── register/
│   │   └── auth.routes.ts
│   ├── dashboard/
│   │   ├── dashboard.component.ts
│   │   └── dashboard.routes.ts
│   ├── profile/
│   │   └── profile.component.ts
│   ├── accounts/                      # COA tree
│   │   ├── coa-list/
│   │   └── coa-form/
│   └── transactions/                  # Phase 2 – placeholder routes
│       └── coming-soon/
├── layout/
│   ├── main-layout/                   # sidebar + topbar + router-outlet
│   └── auth-layout/                   # centered card for login/register
├── app.routes.ts
└── app.config.ts
```

2.4 Environment

src/environments/environment.ts :

```
export const environment = {  
  production: false,  
  apiUrl: 'http://localhost:5027',  
  googleClientId: 'YOUR_GOOGLE_OAUTH_CLIENT_ID.apps.googleusercontent.com'  
};
```

2.5 CORS (backend — add before Angular dev)

Backend currently has **no CORS**. Add in Program.cs before UseAuthentication :

```
builder.Services.AddCors(options =>  
{  
  options.AddPolicy("AngularDev", policy =>  
    policy.WithOrigins("http://localhost:4200")  
      .AllowAnyHeader()  
      .AllowAnyMethod());  
});  
  
// ...  
  
app.UseCors("AngularDev");
```

3. Core Angular implementation

3.1 Models (match API)

```
// api-response.model.ts  
export interface ApiResponse<T> {  
  success: boolean;  
  statusCode: number;  
  message: string;  
  data: T;  
  meta?: { totalData: number; totalPages: number } | null;  
}  
  
// auth.model.ts  
export interface LoginRequest {  
  userNameOrEmail: string;  
  password: string;  
}  
  
export interface AuthUser {  
  id: number;  
  userName: string;  
  email: string;
```

```

    firstName: string;
    lastName?: string | null;
    currencyCode: string;
  }

  export interface AuthTokenData {
    accessToken: string;
    refreshToken: string;
    expiresIn: number;
    user: AuthUser;
  }

  // coa.model.ts - matches Domain Coa JSON
  export interface Coa {
    id: number;
    userInfoId: number;
    parentId?: number | null;
    accountId: number;
    accountType?: { id: number; code: string; name: string; normalBalance: string };
    accountCode: string;
    accountName: string;
    isSystemDefault: boolean;
    isActive: boolean;
    createdAt: string;
    updatedAt?: string | null;
  }

```

3.2 ApiService (unwrap envelope)

```

get<T>(url: string) {
  return this.http.get<ApiResponse<T>>(`${environment.apiUrl}${url}`)
    .pipe(map(res => {
      if (!res.success) throw res;
      return res.data;
    }));
}

```

Show `message` from envelope in toast/snackbar on success/error.

3.3 AuthInterceptor (Bearer + refresh)

1. Clone request with `Authorization: Bearer ${accessToken}`
2. On **401** (not login/refresh): call refresh once, retry request
3. On refresh failure: logout and navigate to `/login`

Store tokens in `sessionStorage` or `localStorage` (prefer `sessionStorage` for access, `localStorage` for refresh if "remember me").

3.4 Routes

```
export const routes: Routes = [
  {
    path: 'auth',
    component: AuthLayoutComponent,
    canActivate: [guestGuard],
    children: [
      { path: 'login', loadComponent: () => import('./features/auth/login/...') },
      { path: 'register', loadComponent: () => import('./features/auth/register/...') },
      { path: '', redirectTo: 'login', pathMatch: 'full' }
    ]
  },
  {
    path: '',
    component: MainLayoutComponent,
    canActivate: [authGuard],
    children: [
      { path: 'dashboard', loadComponent: () => import('./features/dashboard/...') },
      { path: 'profile', loadComponent: () => import('./features/profile/...') },
      { path: 'accounts', loadComponent: () => import('./features/accounts/...') },
      { path: '', redirectTo: 'dashboard', pathMatch: 'full' }
    ]
  },
  { path: '**', redirectTo: 'dashboard' }
];
```

4. Feature screens (phase order)

Phase A — Auth (week 1)

Screen	API	UI
Register	POST /api/UserInfos	Form: userName, email, password, firstName, lastName, currency
Login	POST /api/Auths/login	Form + "Sign in with Google"
Google	POST /api/Auths/google	GIS button → send idToken

After register (manual): redirect to login (register does not return tokens).

After Google: tokens returned → go to dashboard.

Phase B — Layout + Profile (week 1–2)

Screen	API
Profile	GET/PUT /api/UserInfos/{id} — id from JWT user

Sidebar: Dashboard, Accounts, Profile, Logout.

Phase C — Chart of Accounts (week 2)

Screen	API
--------	-----

COA tree	GET /api/Coas — build tree client-side from parentId
Add account	POST /api/Coas
Edit	PUT /api/Coas/{id}
Delete	DELETE /api/Coas/{id} — block if isSystemDefault

Load account types from GET /api/AccountTypes for dropdown.

Phase D — Dashboard v1 (week 2–3)

See Section 5 below.

Phase E — Transactions (when backend ready)

Income / Expense / Transfer forms → voucher engine APIs (future).

5. Project Dashboard — design spec

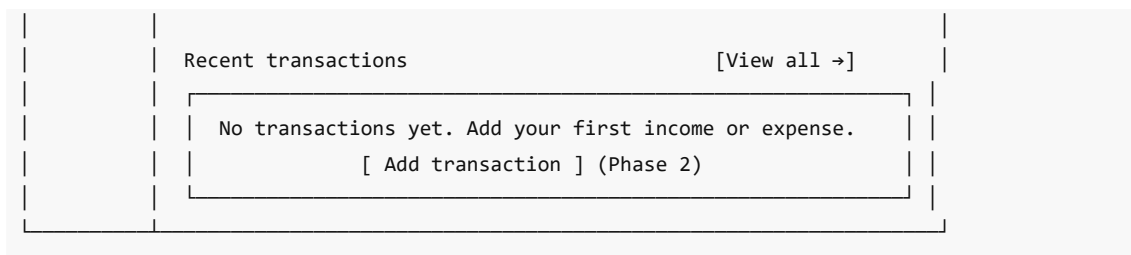
5.1 Purpose

First screen after login. Gives **at-a-glance finance summary** and **quick actions**.

Until Transaction APIs exist, show **real COA data** + **placeholder metrics**.

5.2 Layout wireframe

[≡] FinTrack   Hridoy (BDT ▼) [Logout]				
Dashboard Accounts Profile	Good morning, Hridoy! Aug 27, 2026			
	Total Balance	Income This Month	Expense This Month	Net This Month
	₳ –	₳ –	₳ –	₳ –
	(Phase 2)	(Phase 2)	(Phase 2)	(Phase 2)
Quick actions				
[+ Income] [+ Expense] [⇄ Transfer] (disabled Phase 2)				
Account balances (live)		Expense breakdown		
• Cash		(chart – Phase 2)		
• Bank				
• Mobile Wallet				
• Credit Card				
(from COA 10100–10300, 20100 – balances Phase 2)				



5.3 Dashboard sections (component breakdown)

Section	Component	Data source (now)	Data source (later)
Header greeting	DashboardHeaderComponent	JWT user firstName, currencyCode	—
Summary cards (4)	SummaryCardsComponent	Placeholder –	Reports API
Quick actions	QuickActionsComponent	Buttons disabled + tooltip	Transaction APIs
Account balances	AccountBalancesComponent	GET /api/Coas filter asset/liability leaf accounts	Balance from ledger
Expense chart	ExpenseChartComponent	Empty state	Monthly report API
Recent transactions	RecentTransactionsComponent	Empty state	GET /api/Transactions? limit=5

5.4 Dashboard — Phase 1 (implement now)

1. Load user from token / GET /api/UserInfos/{id}
2. Load COAs → filter codes 10100 , 10200 , 10300 , 20100
3. Show account **names** in balances list; amount shows – until backend posts transactions
4. Summary cards show – with subtitle “Coming soon”
5. Material card grid, responsive (4 → 2 → 1 columns)

5.5 Dashboard — Phase 2 (when backend has transactions)

Card	Formula
Total Balance	Sum asset balances – liabilities
Income This Month	Sum INCOME transactions current month
Expense This Month	Sum EXPENSE transactions current month
Net This Month	Income – Expense

Charts: pie (expense by category), line (6-month trend).

5.6 Color & UX tokens (suggested)

Token	Use
Primary #1565C0	Headers, primary buttons

Income #2E7D32	Positive amounts
Expense #C62828	Negative amounts
Surface #FAFAFA	Page background
Card #FFFFFF	Summary cards

Currency: use `CurrencyPipe` with user's `currencyCode` (default BDT).

6. Screen → API checklist

Register

```
POST /api/UserInfos
{
  "userName": "hridoy",
  "email": "hridoy@example.com",
  "password": "Secret@123",
  "firstName": "Hridoy",
  "lastName": "Ahmed",
  "currencyCode": "BDT"
}
```

Login

```
POST /api/Auths/login
{ "userNameOrEmail": "hridoy", "password": "Secret@123" }
```

Google

```
POST /api/Auths/google
{ "idToken": "<from Google Sign-In>" }
```

Dashboard (Phase 1)

```
GET /api/UserInfos/{id}    Authorization: Bearer ...
GET /api/Coas              Authorization: Bearer ...
```

7. Implementation timeline (recommended)

Week	Deliverable
1	Project scaffold, environments, CORS, ApiService, Auth (login/register/logout/refresh), guards
2	Main layout, sidebar, profile page, COA tree page

3	Dashboard v1 (live COA names + placeholders), polish, error toasts
4+	Transactions UI when backend Phase 2 ships

8. Quality & conventions (match backend)

- **No magic strings/numbers** — use `constants/` folder (API paths, COA codes, storage keys)
 - **Feature folders** — one folder per controller (`auth` , `user-infos` , `coas` , `account-types`)
 - **Classic constructor** + private fields in services (match backend controller style)
 - **Optional fields** — TypeScript `? / | null`
 - **Required fields** — non-optional types in interfaces
-

9. Security notes

- Never store password in frontend storage
 - Access token in memory or sessionStorage; refresh in localStorage only if “remember me”
 - Google Client ID is public; secret stays on Google Cloud Console
 - Always use HTTPS in production
 - Logout must call API to revoke refresh token
-

10. Next steps

1. Add **CORS** to FinTrackCore.Api for `http://localhost:4200`
 2. Run `ng new fintrack-web` beside or inside repo (`/client` or `/fintrack-web`)
 3. Implement **Phase A** auth screens
 4. Build **Main layout** + **Dashboard v1** per Section 5
 5. When backend adds Transactions → enable dashboard cards and quick actions
-

Document version: 1.0 — aligned with FinTrackCore backend as of Auth + UserInfo + AccountType + COA.